

Assignment 5 - Data Science

Theoretical Questions

Mohammad Hossein Basouli

June 9, 2025

Question 1

Because there are many things, that could be done in many ways, and one should be creative about choosing the approach that he/she takes to solve the problem. In order for team to approach it effectively, they would have to hold brainstorming sessions, create templates and formulas, as well as revisiting what worked before and how it can be used now.

Question 2

(a) **Explain One-hot, Label, Target, and Hash encoding. Compare them.**

- **One-hot Encoding:** Converts each unique category value into a binary vector. Each category gets its own column, where presence is indicated by “1” and absence by “0”. Suitable for nominal (unordered) categorical variables.
- **Label Encoding:** Assigns each unique category a unique integer value. Suitable for ordinal features but may mislead models with artificial ordering for nominal data.
- **Target Encoding:** Replaces each category with the average value of the target variable for that category. It is supervised and useful for high-cardinality features.
- **Hash Encoding:** Uses a hash function to map category values into a fixed number of buckets. Reduces dimensionality by collisions.

Comparison Table:

Encoding	Type	Dim.	Model	Limitations
One-hot	Nominal	High	All models	High memory usage and sparse representation
Label	Ordinal	Low	Tree-based models	Imposes artificial order on nominal variables
Target	Supervised	Low	Tree-based, linear	High risk of overfitting without regularization
Hash	Nominal	Fixed	All models	Potential for hash collisions, loss of interpretability

- (b) **What are the limitations of One-hot encoding, especially when dealing with high-cardinality categorical features?**
- **High dimensionality:** Each unique category introduces a new feature, leading to very large feature spaces for high-cardinality data.
 - **Sparsity:** Most values in the encoded vectors are zero, which increases storage and computational inefficiency.
 - **Scalability:** Not feasible for real-time or memory-constrained applications with many categories.
- (c) **In what scenarios is label encoding more appropriate than One-hot encoding, and why?**
- When the categorical feature is **ordinal**, and the numeric labels reflect meaningful order (e.g., `small < medium < large`).
 - When the memory is an issue.
- (d) **How can overfitting be avoided when applying target encoding?**
- **Smoothing:** Combine the category mean with the global mean based on category size (number of samples, being in that category) to avoid overfitting.
 - **Cross-validation:** Use out-of-fold target means during training to prevent overfitting.
 - **Noise addition:** Add small random noise to encoded values to regularize and avoid fitting noise in the target.
- (e) **How does hash encoding address the sparsity problem in categorical variables, and what potential issue does it introduce?**
- **Addressing sparsity:** Hash encoding maps categories into a fixed number of buckets using a hash function, reducing the number of features regardless of category count. This mitigates the curse of dimensionality and lowers memory usage.
 - **Potential issue: Hash collisions** occur when different categories map to the same bucket. This can introduce ambiguity and noise, potentially degrading model performance. Also, interpretability is lost as original category names are not preserved.

Question 3

Category embeddings represent categorical variables in machine learning models as continuous vectors in a lower-dimensional space. This method maps each category to a vector, which is learned during model training. The main idea is to embed the categorical variables into a dense, lower-dimensional vector space, rather than representing them as sparse, high-dimensional one-hot vectors.

How Category Embeddings Work:

1. **Initialization:** Each category (value) in a feature is assigned an initial vector, usually initialized randomly.
2. **Learning:** During training, the model adjusts these vectors through backpropagation, learning how to best represent each category such that the resulting embeddings help improve predictive performance.
3. **Representation:** After training, the category is represented as a vector in this learned space, capturing relationships and similarities between categories based on the task at hand.

Why Category Embeddings Outperform Traditional Methods:

1. **Dimensionality Reduction:** One-hot encoding can lead to high-dimensional sparse vectors, especially for features with many categories (high cardinality). Embeddings reduce this dimensionality and map categories into a more compact, dense vector space.
2. **Capturing Similarities:** Embeddings allow the model to learn meaningful relationships between categories. For example, similar categories might be represented by vectors that are closer in space, which isn't possible with one-hot encoding.

Question 4

Benefit: Rounding numerical features can reduce the noise in the data by eliminating small, potentially insignificant variations. This is especially useful when the precise value of a feature is not critical for the prediction task. It can also lead to fewer unique values, which simplifies the model and can reduce overfitting, especially in tree-based models that split on exact thresholds.

Example: $3.1415 \rightarrow 3.14$

Risk:

- **Lossing Important Information:** Rounding can lead to information loss by discarding important fine-grained distinctions between values. This may harm model performance, particularly if the model relies on precise measurements or if small differences in values are predictive.
Risk Example: Rounding temperature from 36.6°C to 37°C may lose medically relevant detail.
- **High Variance Model:** The model would have high variance due to discarding of the noise that might could have helped to make the model robust to the noise.

Question 5

- (a) How can interaction features or polynomial encodings help a linear model solve non-linear problems?

Linear models, by default, can only capture linear relationships between features and the target. However, many real-world problems involve non-linear interactions. To address this, we can apply feature transformations such as:

- **Polynomial Features:** Introducing polynomial terms (e.g., x^2 , x^3) allows a linear model to fit non-linear curves. For instance, adding x^2 enables the model to fit a quadratic relationship.
- **Interaction Terms:** Features like $x_1 \cdot x_2$ capture the combined effect of two variables, allowing the model to account for interactions that are not simply additive.

By augmenting the input space with these derived features, a linear model can approximate non-linear functions through a higher-dimensional linear relationship.

- (b) When engineering features from date/time data, what techniques ensure that models handle cyclical variables correctly?

Date/time components such as hours, days, and months are inherently cyclical. For instance, hour 23 and hour 0 are close in time but numerically far apart. To preserve this cyclical nature, we use:

Sine and Cosine Transformations(Project to a circle, as it says in the slides): Convert cyclical variables using the formulas:

$$x_{\sin} = \sin\left(\frac{2\pi x}{P}\right), \quad x_{\cos} = \cos\left(\frac{2\pi x}{P}\right)$$

where x is the cyclical variable (e.g., hour of day) and P is the period (e.g., 24 for hours). This maps the variable onto the unit circle, preserving its cyclic nature.

Question 6

Role of TF-IDF in Natural Language Processing:

TF-IDF (Term Frequency–Inverse Document Frequency) is a feature extraction technique used to convert textual data into numerical vectors. It reflects the importance of a term in a specific document relative to its prevalence across a corpus. The TF-IDF weight is defined as:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

where:

- $\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}$ is the normalized term frequency.
- $\text{IDF}(t, D) = \log\left(\frac{N}{1 + |\{d \in D : t \in d\}|}\right)$ is the inverse document frequency.

TF-IDF helps emphasize discriminative words that are important in specific documents but rare across others, thereby reducing the influence of commonly occurring, non-informative terms (e.g., stop words). It enables models to operate on sparse, high-dimensional representations of text.

Role of Dimensionality Reduction in Text Data:

TF-IDF representations often result in very high-dimensional sparse matrices, especially when dealing with large vocabularies. Dimensionality reduction techniques are used to:

- **Reduce computational cost:** Lowering the number of features decreases memory usage and training time.
- **Improve generalization:** By eliminating noise and redundant features, dimensionality reduction can reduce overfitting.
- **Reveal latent semantics:** Methods like Latent Semantic Analysis (LSA) or Latent Dirichlet Allocation (LDA) uncover hidden topics or structures in the text.

Common techniques include:

- **Singular Value Decomposition (SVD):** Used in LSA to project the TF-IDF matrix into a lower-dimensional semantic space.
- **Principal Component Analysis (PCA):** Finds orthogonal components capturing the most variance in the data.
- **LDA:** As discussed above.
- **LSA:** As discussed above.

Question 7

Why Neural Networks Require Less Manual Feature Engineering:

Neural Networks, especially deep learning models, often require less manual feature engineering compared to traditional machine learning models due to their ability to perform **automatic feature learning** from raw data. This capability is grounded in the following key reasons:

1. **Hierarchical Feature Learning:** Deep neural networks consist of multiple layers that progressively transform the input data into higher-level, abstract representations. Lower layers capture simple patterns (e.g., edges in images or local n-grams in text), while deeper layers learn complex structures (e.g., objects or semantic meaning).
2. **Non-Linearity and Flexibility:** Through non-linear activation functions (e.g., ReLU, sigmoid, tanh), neural networks can model complex, non-linear relationships that traditional linear models cannot easily capture without manual transformations or interaction terms.
3. **End-to-End Learning:** Deep learning models are typically trained in an end-to-end manner, where the model directly learns the mapping from raw input to output. This reduces the need for domain-specific preprocessing or handcrafted features.
4. **Representation Learning:** Neural networks learn internal representations (embeddings) of data that are optimized for the task (e.g., classification or regression). For instance, word embeddings in NLP capture semantic similarity without manual encoding.
5. **Reduced Dependence on Domain Expertise:** Traditional models often rely on domain knowledge to design effective features. Neural networks alleviate this burden by learning features automatically from large datasets.

Conclusion:

While traditional machine learning models (e.g., decision trees, logistic regression, SVMs) typically require extensive manual feature engineering to perform well, neural networks can learn rich, task-specific features directly from raw inputs. This makes them particularly well-suited for unstructured data such as text, images, and audio.